AudioSight — Project Profile Document

AMD AI DevMaster Hackathon — Track 1: Development of Multimodal Content Creation Tools

Item	Value
Application name	AudioSight
Track	Track 1 — Multimodal AI / Content Creation Tools
Participant	`austin1997`
Version	v0.1
Platform	Linux x86_64, Python 3.12
Verified hardware	AMD Radeon (gfx1100, RDNA 3), 48 GiB VRAM
Verified software	ROCm 7.2.1, PyTorch 2.10.0+rocm7.2.1, vLLM 0.18.1
Source repository	https://github.com/austin1997/AudioSight
Submission repository	https://github.com/austin1997/Radeon-hackathon-2026-07
Demo video	https://github.com/austin1997/Radeon-hackathon-2026-07/blob/main/demo.mp4

The submission repository is a fork of the contest repository. It contains the four required deliverables and references the source repository as a git **submodule** (`AudioSight/`) rather than a copied snapshot, so the code under review is always an exact upstream commit rather than a divergent duplicate. Clone it with `git clone --recurse-submodules`; the submodule pointer is published once the source repository is opened to the public.

---

1. Project Background

Multimodal AI has advanced at remarkable speed, but almost all of that progress points in one direction: **towards the eye**. Text-to-image, text-to-video, image editing, style transfer, virtual try-on, digital humans — the generative ecosystem is overwhelmingly about producing things to look at, and we now have an abundance of excellent open models and tools for doing exactly that.

Somewhere in that rush, we seem to have forgotten that a substantial number of people cannot see any of it. According to the World Health Organization, at least 2.2 billion people live with a vision impairment. For them, a film, a TV episode, or a beautifully rendered game cinematic is a stream of dialogue and sound effects with unexplained silences in between — the visual half of the story simply does not arrive.

The established solution is **audio description (AD)**: a narrator describes the actions, settings, characters, and on-screen text during natural pauses in the dialogue. It works extremely well, and it is also expensive, slow, and manual. As a result only a small fraction of the world's video catalogue has an audio description track, and for languages other than English the coverage is much worse.

AudioSight exists to close that gap. It is an offline, local-GPU command-line tool that takes an ordinary video file and produces a complete audio-described soundtrack: it listens to the dialogue, watches the picture, understands what is happening scene by scene, writes narration in

Chinese and English, speaks that narration, and mixes it into the original audio at moments where it will not collide with speech.

The goal is simple to state: **people with visual impairments should be able to enjoy great visual works too** — television, film, and in time, video games. The same wave of multimodal research that gave us today's image and video generators has also produced openly available models that can read a video rather than write one: models that understand scenes, transcribe speech, and synthesize a natural voice. AudioSight is built on those models, turning the visual half of the story back into something a blind or low-vision audience can experience.

2. Target Users & Application Scenarios

AudioSight is built for two groups of users, and it deliberately serves them with the same pipeline under two different review policies.

2.1 People with visual impairments (direct use)

A visually impaired user — or a family member, teacher, or volunteer helping them — can point AudioSight at a video they already own and get back an audio file they can listen to on any player. There is no human proofreading step in this mode, so the narration is machine-generated and will not match the polish of a professionally scripted AD track. It does not need to. The alternative today is usually nothing at all, and a described soundtrack that is substantially correct lets the listener follow who is on screen, where the scene takes place, what characters are doing, and what on-screen text says.

Because the entire pipeline runs locally and offline after the models are downloaded, personal media never leaves the machine — an important property when the source is a purchased film, a family video, or recorded course material.

```
audiosight init lecture.mkv --project work/lecture --review-policy prototype-auto
audiosight analyze work/lecture
audiosight draft   work/lecture --languages zh-CN,en-US
audiosight validate work/lecture --language zh-CN
audiosight render  work/lecture --language zh-CN --output outputs/lecture.m4a
```

2.2 Accessibility content creators (assisted authoring)

Professional and volunteer AD producers are the second audience, and for them AudioSight is an acceleration tool rather than a replacement. The default `manual` review policy stops the pipeline after drafting and hands the creator a structured YAML file: one entry per narration line, each with a sample-accurate time anchor, the model's confidence, an explicit list of uncertainties, and four evidence frames extracted from the scene it describes. The creator edits the text, listens to any single line in context with `audiosight preview`, approves or rejects entries, and only then renders.

This turns AD authoring from "watch the whole film and write a script from scratch" into "review and correct a draft", which is a fundamentally cheaper task. Every render report records whether a human actually reviewed the content (`human_reviewed`), so machine drafts can never be silently published as human-verified work.

2.3 Scenarios explicitly in scope

- Films, TV episodes, documentaries, and short films with or without subtitles.

- Bilingual output: an English-language source can be rendered with Chinese narration, English narration, or both, from a single analysis pass.
- Educational and museum video that must meet accessibility requirements.
- Batch pre-processing of a back catalogue to give human describers a starting draft for every title.

2.4 Future direction

The architecture — perceive, understand, describe, speak, mix — is not specific to pre-recorded video. Extending the same worker model to game capture and eventually to real-time engine hooks would let visually impaired players follow what is happening on screen in a video game, which is currently one of the least accessible forms of visual media.

3. System Architecture

The pipeline has three stages, which correspond to the three main CLI commands. `analyze` extracts the soundtrack and frames, transcribes the dialogue, splits the video into scenes, and records the visible facts of each scene together with a running summary of the story so far. `draft` turns those facts into scene narration, optionally stopping for human review. `render` speaks the narration and appends it to the scene soundtrack, producing the final composited audio track. The rest of this section describes how that flow is isolated across processes. A rendered diagram of the same pipeline is in the repository README.

3.1 Design principle: an isolated core, short-lived GPU workers

The single most important architectural decision in AudioSight is that the **core process contains no machine-learning framework at all**. It never imports PyTorch, Transformers, vLLM, or any Qwen package. It owns configuration, project state, media operations, review truth, validation, and rendering.

All neural inference happens in **short-lived worker processes** launched into separate, self-contained Python environments and addressed through a versioned JSON request/response protocol:

```
audiosight-core            ── no Torch, no CUDA/ROCm, no model code
  │  writes request.json, runs: <env-python> -m audiosight.worker_main
  │
  ├─▶ audiosight-vl        ── Torch 2.10 (ROCm) + vLLM 0.18.1 + TorchCodec
  │      asr/analyze  vision/cluster  vision/analyze  narration/draft
  │
  └─▶ audiosight-tts       ── same Torch/vLLM ABI + vLLM-Omni 0.18.0
         tts/synthesize_batch
```

This buys three things that matter a great deal on a single-GPU machine. Each model gets the whole GPU while it runs and fully releases it on worker exit, so a 48 GiB card can host a 9B multimodal model, an ASR stack, and a two-stage TTS pipeline in one job without ever holding them resident together. The dependency graphs of vLLM, vLLM-Omni, and TorchCodec — which conflict — are kept in separate environments. And the core CLI stays installable and testable on a machine with no GPU at all.

3.2 Command flow



Commands never implicitly run their prerequisites. `draft` will not silently re-analyze, `render` will not re-draft or touch review files, and `validate` never repairs human-edited text. Every stage transition is recorded in a WAL-mode SQLite state database, protected by a project lock, and every persistent artifact is written atomically.

3.3 The `analyze` pipeline

`analyze` is where the multimodal work happens. Its ordered sub-stages, and where each one executes:

#	Sub-stage	Device	What it does
1	Audio preparation	CPU / FFmpeg	Extracts a 48 kHz stereo float32 render master and a 16 kHz mono ASR track, both padded to the exact source timeline.
2	Shot detection	CPU	Builds a 5 fps, 480 px proxy and runs PySceneDetect `ContentDetector` to cut the film into raw shots.
3	Candidate construction	CPU	Normalizes shots into gap-free candidate segments, splitting over-long shots and gluing very short ones forward.
4	ASR + forced alignment	**GPU**	Qwen3-ASR transcribes the dialogue through vLLM; Qwen3-ForcedAligner then produces word-level timestamps.
5	Scene clustering	CPU **+ GPU**	Deterministic visual/dialogue features decide most shot boundaries; only genuinely ambiguous ones are escalated to the multimodal model.
6	Evidence & clip extraction	CPU / FFmpeg	Writes four evidence JPEGs and one width-limited video clip per merged scene.
7	Visual analysis	CPU decode + **GPU**	Qwen3.5 watches each scene clip and emits structured visible facts, inferences, uncertainties, burned-in subtitles, entities, and locations, updating a running story state.

Stage 5 is the part we consider most interesting, and it is described in detail in section 4.3: asking a 9B model about every one of ~80 shot boundaries would be wasteful, so the pipeline only pays for semantic judgement where cheap signal processing is genuinely undecided.

Every stage is content-addressed and checkpointed. Cache keys include the action-specific protocol version, the model revision, the generation settings, and a digest of the worker environment, so changing a prompt, a sampling parameter, or a dependency correctly invalidates only what it should. A worker killed halfway through visual analysis resumes from its last checkpoint rather than restarting the scene sweep.

3.4 Drafting, review, and rendering

`draft` feeds the transcript, the visual events, and the accumulated story state to Qwen3.5 in eight-scene chunks and produces one narration item per scene, per language, with a duration-aware length budget. `validate` then checks the reviewed YAML against the timeline — anchors must not fall inside a spoken word, text must be non-empty, referenced evidence must exist — and writes a report bound to the SHA-256 of the exact review file it inspected. `render` refuses to run without a matching valid report.

Rendering uses an **expanded-timeline** model rather than fitting narration into existing silence. At each anchor the original soundtrack is paused, a short lead-in silence, the synthesized narration, and a lead-out silence are inserted, and the original audio resumes exactly where it stopped:

```
… source audio … | 150 ms | narration | 150 ms | … source audio continues …
                 ▲ anchor, nudged clear of any aligned word
```

Nothing is overlaid or ducked, so no dialogue is ever masked and the description is never truncated to fit a gap that is too short. The cost is a longer running time, which is a well-established trade-off in the AD community and the right default for v0.1. Source gain is measured with FFmpeg's EBU R128 filter and normalized to −18 LUFS with a −1.5 dBFS true-peak ceiling. Output is 48 kHz stereo WAV or AAC-LC at 192 kbit/s, and a JSONL map records the sample-accurate correspondence between source and output timelines in both directions.

4. Model & Algorithm Introduction

4.1 Models

All four models are Apache-2.0 licensed, run locally, and are pinned to an exact revision in `models/manifest.lock.json`.

Role	Model	Runtime	Purpose
ASR	`Qwen/Qwen3-ASR-1.7B`	vLLM 0.18.1	Dialogue transcription and source-language detection.
Alignment	`Qwen/Qwen3-ForcedAligner-0.6B`	Transformers SDPA	Word-level timestamps.
Vision + text	`Qwen/Qwen3.5-9B` (default) or `Qwen/Qwen3.5-4B`	vLLM 0.18.1	Scene clustering, visual analysis, and narration drafting.
TTS	`Qwen/Qwen3-TTS-12Hz-1.7B-CustomVoice`	vLLM-Omni 0.18.0	Narration speech synthesis.

Three hardware profiles select the vision/text model and all associated memory ceilings and batch sizes:

Profile	Model	Free VRAM required	Worker cap	Batch sizes
`workstation-32gb` (default)	Qwen3.5-9B bf16	29.5 GiB	30 GiB	4
`portable-16gb`	Qwen3.5-4B bf16	14.5 GiB	15 GiB	1
`portable-16gb-q4` (experimental)	Qwen3.5-9B W4A16 GPTQ	14.5 GiB	15 GiB	1

Qwen3.5's decoder is a hybrid of 24 Gated DeltaNet layers and eight full-attention layers, which is directly relevant to the ROCm kernel selection discussed in section 5.

4.2 Speech understanding

Rather than cutting the audio at fixed 30-second boundaries, the ASR worker searches for the lowest-energy point near each 30-second mark and splits there, which keeps words intact across chunk edges. Up to four chunks are submitted to vLLM in one generation call for continuous

batching. Forced alignment then runs as a batched Transformers SDPA pass — vLLM does not support the aligner's token-classification architecture — producing per-word `start_ms` / `end_ms`.

Those word timings are the backbone of everything downstream. They tell the clustering stage whether a sentence bridges a shot boundary, and they tell the renderer exactly which milliseconds are unsafe for an insertion.

4.3 Scene clustering: cheap signals first, the model only when undecided

A ten-minute episode yields roughly 80 shots, but far fewer scenes. Merging shots into semantically coherent scenes matters because narration should describe a scene, not a camera angle. Asking a 9B multimodal model about all 78 boundaries would dominate the runtime.

So the core process first computes two purely deterministic signals per boundary, on the CPU:

- **Visual similarity.** Frames sampled just inside each candidate are compared using a 64-bit grayscale difference hash and a 32×32 HSV hue/saturation histogram correlation, combined as `0.45 × dhash + 0.55 × histogram`. Only the upper 62 % of each frame is compared, which keeps burned-in subtitles from dominating the score.
- **Dialogue continuity.** Using the word-level alignment, does a spoken word cross the boundary, and how large is the gap between the last word before and the first word after? The default continuity threshold is 1200 ms.

These resolve the clear cases immediately: similarity ≥ 0.82 with continuous dialogue merges, similarity ≥ 0.92 merges regardless, and similarity ≤ 0.45 without a dialogue bridge splits. Everything left over is marked `REQUIRES_SEMANTIC_JUDGMENT`, and only those boundaries are sent to Qwen3.5 with three-second clips from either side plus the surrounding dialogue. Malformed or missing decisions fall back conservatively to a split, affecting only that one boundary. Post-processing then enforces the 6–45 second scene duration window.

4.4 Visual analysis and structured decoding

Scene clips are decoded on the CPU with TorchCodec, which materializes only the frame indices implied by the configured 2 fps sampling and frame limits, and hands vLLM the frame tensors together with the native FPS and original frame indices so the model retains correct relative timestamps.

Every model response — clustering decisions, visual-analysis objects, narration plans — is constrained by an **xgrammar JSON schema** at decode time. This replaced an earlier design that used Python token-by-token stopping criteria followed by a JSON-repair generation pass, and it eliminates malformed output as a class of failure rather than handling it after the fact. When a completion is truncated by the length limit, the worker salvages fully decoded observations, records `VISION_STRUCTURED_LENGTH_FALLBACK`, and retries that scene through a reduced facts-only schema instead of losing it.

Scene analysis is deliberately **serial**: each scene's result updates the story state (who has been introduced, where we are, what has already been explained) that conditions the next scene. This is what stops the narration from re-introducing the same character in every scene. Clustering and narration drafting have no such dependency and are batched.

Both clustering and analysis run with thinking disabled and Qwen3.5's official Instruct sampling configuration (`temperature=0.7`, `top_p=0.8`, `top_k=20`, `presence_penalty=1.5`). Narration thinking is implemented behind the Qwen3 reasoning parser but remains off by default: on an identical 9B analysis it took 1623 s versus 461 s and produced only 11/12 items per language instead of 19/19.

4.5 Narration length and speech synthesis

Narration length is budgeted per scene from its duration, the target language, and a `sparse` / `standard` / `detailed` density setting — 1.4 Han characters or 0.85 English words per second at standard density, clamped to sane minimums and maximums. Validation reports every soft-limit miss as a warning rather than silently truncating the text.

TTS runs through the vLLM-Omni two-stage Talker/Code2Wav pipeline. Output is normalized to 48 kHz mono, and playback rate is applied with FFmpeg `atempo` (the `narration_speed` setting, 3.0 by default for the 12 Hz model). If Omni fails to initialize or generate, AudioSight records an explicit `TTS_VLLM_OMNI_UNAVAILABLE` code before falling back to the qwen-tts SDPA backend — the backend change is always visible in the report, never silent.

5. Adaptation for AMD Radeon GPU / ROCm

AudioSight was brought up and verified end to end on an **AMD Radeon gfx1100 (RDNA 3) GPU with 48 GiB VRAM running ROCm 7.2.1**. ROCm is not a secondary target that happens to work; it is the platform the entire pipeline was measured and tuned on.

5.1 The ROCm software stack

Component	Version	Source
ROCm	7.2.1	system
PyTorch	2.10.0+rocm7.2.1	`repo.radeon.com` official wheel
Triton	3.6.0+rocm7.2.1	`repo.radeon.com` official wheel
torchvision / torchaudio	0.25.0 / 2.10.0 +rocm7.2.1	`repo.radeon.com` official wheels
vLLM	0.18.1	built from source for `gfx1100`
vLLM-Omni	0.18.0	built against the same ABI
TorchCodec	0.10	built from source against ROCm Torch
Transformers	4.57.6	PyPI
amdsmi	7.0.2	PyPI

Three of these required real porting work.

vLLM is compiled from source for RDNA 3. There is no prebuilt ROCm wheel for vLLM 0.18.1 that targets gfx1100, so `scripts/create-rocm-envs.sh` downloads the pinned sdist and builds it with `PYTORCH_ROCM_ARCH=gfx1100` and `VLLM_TARGET_DEVICE=rocm`. Because `audiosight-vl` and `audiosight-tts` are pinned to a byte-identical Torch ABI, the compiled source tree is built once and shared by both environments, which roughly halves setup time and guarantees the two runtimes cannot drift apart.

TorchCodec is compiled from source. The generic TorchCodec CPU wheel on PyPI is not ABI-compatible with the ROCm PyTorch build and fails at import. The setup script builds the pinned TorchCodec 0.10 revision against Torch 2.10 and the system FFmpeg development libraries, installing those libraries automatically when they are missing.

A ROCm-specific FlashAttention false positive is guarded. The verified ROCm base image ships an unusable optional CUDA `flash_attn` package. The bundled vLLM revision probes only for the package root, concludes FlashAttention is available, and then fails. AudioSight explicitly

guards that probe so vLLM selects its proper ROCm fallback. FlashAttention is not a runtime requirement of this project.

5.2 Kernel selection on gfx1100

PyTorch exposes HIP devices through the `torch.cuda` API, so workers use explicit `cuda:0` placement while reporting `backend: rocm`. On the verified system, vLLM selects a different attention path for each part of the workload:

- Qwen3.5 full-attention layers → `TRITON_ATTN`
- Qwen3.5's 24 Gated DeltaNet layers → **Triton / FLA kernels**
- Qwen3.5 vision tower → **Torch SDPA**
- Qwen3-ASR generation → vLLM; forced alignment → Transformers SDPA
- Qwen3-TTS → vLLM-Omni, eager execution on both stages

Compilation artifacts are keyed by action and model revision and cached under `models/.runtime-cache/`, so the one-time Triton/vLLM compilation cost is paid once rather than on every worker launch.

5.3 Strict single-GPU discipline

Several rules are enforced in code rather than left to configuration, because each of them silently destroys performance on a single-GPU workstation:

- No `device_map="auto"`, no automatic sharding.
- No CPU or disk offload: vLLM runs with swap space and CPU offload set to zero.
- **No silent CPU fallback.** If a model cannot be placed on the GPU, the worker fails with a diagnostic exit code instead of quietly running at 1 % speed.
- Model parameters and input tensors are asserted to be on the GPU during `doctor --deep`.
- All model access is by repository-local path with Hugging Face offline flags set; no global cache, no runtime network access.

`audiosight doctor --accelerator rocm --deep` verifies the GPU vendor, ROCm/HIP compatibility, bf16 support, attention backends, disk space, and then loads each of the four models in turn, runs a minimal real inference, records peak VRAM and duration, and confirms the memory is released on worker exit.

5.4 Measured VRAM behaviour

Deep smoke-test peaks on gfx1100, one model resident at a time:

Model	Peak
Qwen3-ASR via vLLM	25 936 MiB
Forced aligner (SDPA)	1 835 MiB
Qwen3.5-9B bf16	25 607 MiB
Qwen3.5-4B bf16	11 240 MiB
Qwen3-TTS via vLLM-Omni (both stages)	26 130 MiB

The TTS figure is sampled from total device usage because Omni starts a separate EngineCore process per stage; the parent process's PyTorch allocator statistics alone would incorrectly

report zero. The 4B numbers confirm the `portable-16gb` profile is genuinely viable on a 16 GiB Radeon card.

5.5 Throughput optimization: continuous batching

The largest ROCm-specific performance win came from restructuring independent model requests so vLLM's continuous batching can overlap them. Measured end-to-end worker times (including model load) on fixed analysis artifacts:

Workload	Batch 1	Batch 2	Batch 4	Batch 8
Qwen3.5-9B clustering, 16 boundaries	136.810 s	118.798 s	**104.029 s**	101.781 s
Qwen3.5-9B narration draft, en-US	132.581 s	110.477 s	**101.225 s**	—
Qwen3.5-9B narration draft, zh-CN	117.060 s	101.715 s	**98.566 s**	—
Qwen3-ASR, 21 chunks + alignment	49.607 s	41.607 s	**37.296 s**	34.780 s

Batch 4 is the default for `workstation-32gb`: it delivers 24.0 % on clustering, 23.7 % on English drafting, and 24.8 % on ASR, while batch 8 adds only ~2 % — below the 5 % threshold we set for adopting a change, and for ASR it perturbed more forced-alignment timestamps. Peak VRAM barely moves (clustering: 25 319 MiB at batch 1 versus 25 789 MiB at batch 4), so the throughput is essentially free. On the portable profile the same change yielded only 11–13 %, below the 15 % rollout threshold, so portable concurrency deliberately stays at 1.

Crucially, all batch settings returned the **same 16 clustering decisions** with no fallbacks and identical prompt-token counts. Batching here is a pure throughput change, not a quality trade.

The earlier migration from Transformers to vLLM produced a comparable win at the kernel level: an identical four-boundary generation call fell from 21.376 s to 12.326 s, and decoder throughput rose from roughly 9.5–15.6 tokens/s to 18.3–40.2 tokens/s.

5.6 TTS backend comparison

Both TTS backends were measured on the same 19 reviewed narration items, including startup, synthesis, normalization, mixing, and AAC encoding:

Language	qwen-tts (SDPA)	vLLM-Omni (batch 4)	Change
en-US	415.505 s	234.964 s	**43.5 % faster**

Continuous batching clearly wins on this workload, but vLLM-Omni's two-stage startup and its Code2Wav graph capture are a fixed cost that can outweigh the gain on shorter inputs. This is exactly why the Omni path is labelled experimental and why any fallback is recorded as `TTS_VLLM_OMNI_UNAVAILABLE` rather than silently switching backends.

5.7 What runs on the CPU, and why we say so

FFmpeg clip transcoding, audio resampling, filtering, concatenation, and AAC encoding run on the CPU, as does TorchCodec's frame decoding and the deterministic shot detection. AMD's hardware video codec engines do not accelerate these operations in the baseline configuration. We state this explicitly rather than implying the whole pipeline is GPU-resident: the GPU does all four neural workloads, and nothing else pretends to.

5.8 End-to-end verification on Radeon

A full bilingual run on a 601.44-second (10:01) source, entirely on gfx1100:

Result	Qwen3-VL baseline	Qwen3.5-9B	Qwen3.5-4B
Shots / candidates / boundaries	80 / 79 / 78	80 / 79 / 78	80 / 79 / 78
Merged scenes	21	19	18
Transcript words	842	842	842
Visible facts extracted	177	**343**	196
Analyze wall time	953 s	2057 s	1713 s
Bilingual narration draft	114 s	461 s	216 s

All three runs produced a byte-identical transcript (SHA-256 `f22443b2…`), which is a useful cross-check that the ASR/alignment path is deterministic across model changes on ROCm. Every language passed validation with zero errors and rendered to M4A. The move to Qwen3.5 roughly doubled the number of visible facts extracted from the same footage, which is the metric that most directly determines how much a blind listener actually learns from the description.

6. Deliverables

- **Source code**: complete repository with a `src/` layout, strict mypy, Ruff linting, and a deterministic test suite runnable with no GPU and no model weights via `AUDIOSIGHT_WORKER_MODE=fake`, at https://github.com/austin1997/AudioSight and linked into the submission repository as the `AudioSight/` git submodule.
- **README**: prerequisites, ROCm environment setup, model fetching, full startup guide, complete dependency list, and troubleshooting.
- **Reproducible environments**: `scripts/create-rocm-envs.sh` plus pinned requirement files with wheel URLs and SHA-256 hashes for every ROCm artifact.
- **Output**: 48 kHz stereo audio-described soundtracks in Simplified Chinese and English, plus machine-readable validation/render reports and a sample-accurate source↔output timeline map.
- **Demo video**: end-to-end CLI operation on AMD Radeon hardware.

7. Acknowledgements

We would like to thank **AMD** for supporting this project.

This work exists because AMD made Radeon GPU resources and the ROCm software stack available to independent developers through the AI DevMaster Hackathon. Every model in this pipeline — speech recognition, forced alignment, multimodal scene understanding, narration planning, and speech synthesis — was brought up, debugged, benchmarked, and verified on AMD Radeon hardware. An open, capable compute platform is what makes it possible for a small team to build something like this at all.

We are especially grateful that AMD's programme gave room for a project aimed at people with visual impairments. Accessibility work rarely has a commercial sponsor, and the visually impaired community is too often left out of the conversation about what AI should be built for. Backing a tool whose only purpose is to let blind and low-vision audiences enjoy films, television, and one day games says something genuinely good about AMD's priorities — and it is appreciated far beyond this hackathon.

Thanks also to the Qwen team for releasing capable multimodal, ASR, and TTS models under Apache-2.0, and to the vLLM and PyTorch ROCm communities whose work made a single-GPU pipeline of this scope practical.